

1.

$d(n)$ is the number of positions where the current word is different from the goal word.

Example: current word dog; goal cat, $d(n) = 3$

Steps:

Cost: $h(n) = d(n)$

Pseudocode:

Different = 0

For every corresponding pair of letters:

 If letters are different:

 Different += 1

Return different

It can not overestimate because each action can only change one letter, and if there are d incorrect positions, at least d actions are needed, and each action costs 1. Then the true remaining cost must be at least d .

Scrabble:

Cost: $h(n) = \text{sum of Scrabble value of each incorrect goal letter}$

Pseudocode:

Heuristic = 0

For each position i in the word:

 If $\text{current}[i] \neq \text{goal}[i]$:

 Heuristic = heuristic + scrabble_value(goal[i])

Return heuristic

It can not overestimate because every position that differs from the goal must have the correct goal letter inserted eventually. The heuristic only adds the Scrabble cost of those needed goal letters. If other steps are needed, they can only add more cost, so the heuristic will not exceed the minimum remaining cost.

Frequency:

Cost: $m = \text{smallest rarity value in the dictionary}$, $h(n) = d(1+m)$

Pseudocode:

Different = 0

For each position i in the word:

 If current[i] != goal[i]:

 Different = different + 1

Smallest = smallest rarity value in the dictionary

Return different * (1+smallest)

It can not overestimate because each remaining action costs at least 1 + the smallest rarity value in the dictionary. If d letters are incorrect, at least d actions are needed, and the remaining cost will be at least d(1+ smallest rarity value).

2.

Word Pair	Cost Measure	UCS Cost	UCS Nodes Expanded	A* Cost	A* Nodes Expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.93%
WHY → ASK	Scrabble	9	765	9	13	98.30%
WHY → ASK	Frequency	38.8554	808	38.8554	754	6.68%
BEAR → DUCK	Steps	4	691	4	6	99.13%
BEAR → DUCK	Scrabble	11	2981	11	6	99.80%
BEAR → DUCK	Frequency	38.2402	1084	38.2402	714	34.13%

1. Yes, UCS and A* always returned the same optimal cost.
2. For WHY -> ASK, the Scrabble heuristic produced the largest reduction of 98.30%. For BEAR -> DUCK, the Scrabble heuristic also produced the largest reduction: 99.80%.
3. For WHY -> ASK, the frequency heuristic was the least informative, with a 6.68% reduction. For BEAR -> DUCK, the frequency heuristic was also the least informative, with a 34.13% reduction.
4. The steps and scrabble heuristics were more informative because they estimate unavoidable costs. The steps counts the number of incorrect letters and the scrabble adds the scrabble values of the goal letters that still need to be inserted. The frequency heuristic showed to be less informative because it uses the minimum rarity value found in the whole dictionary. This minimum could be much smaller than the rarity cost of the words in the solution path and this would cause the heuristic to underestimate the remaining cost. This explains why A* had a much smaller reduction for frequency in both of the problems.

3.

Steps: Yes, the optimal cost from w_1 to w_2 must always be the same as the optimal cost from w_2 to w_1 . This is because legal actions in steps all cost 1, so the path will cost the same if reversed.

Scrabble: No, the cost of a move is determined by the letter that is being inserted into the word. If the move is reversed, a different letter is inserted, and that letter may have a different cost. An example would be going from dog to cog inserts c, which costs 3, but cog to dog inserts d, which costs 2. Even though the move is reversible, the cost will not always be the same.

Frequency: No, the cost of an action is based on the word entered. Therefore, going from one word to another uses the rarity value of the new word, while going in the reverse order uses the rarity value of the original word. If the words have different rarity values, the costs in the reverse directions will also be different. An example of this would be UCS gives an optimal cost of 12.39 from dog to cog, but 6.82 from cog to dog. Even though these actions are reversible, they are not symmetric in cost.

4.

Model Used: Gemini 3.6 Flash

Prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed Heuristic:

1. Proposed Heuristic Definition

Let n be a current word state and g be the goal word state. Let $L(n, g)$ be the set of character positions $i \in \{1, \dots, |n|\}$ where state n differs from state g (i.e., $n[i] \neq g[i]$). The baseline heuristic is $h_0(n) = |L(n, g)|$.

Since every legal action changes exactly one letter, reaching g from n requires at least $h_0(n)$ steps. Therefore, any valid path from n to g must enter at least $h_0(n)$ distinct intermediate or final words.

We define the global minimum entry cost across all valid words in the dictionary D as:

$$c_{\min} = 1 + \min_{w \in D} \text{rarity}(w)$$

Because $\text{rarity}(w) \geq 0$ for all $w \in D$, it holds that $c_{\min} \geq 1$.

We define the rarity-aware heuristic $h_1(n)$ as:

$$h_1(n) = \begin{cases} 0 & \text{if } n = g \\ \text{cost}(g) + (h_0(n) - 1) \cdot c_{\min} & \text{if } n \neq g \end{cases}$$

where $\text{cost}(g) = 1 + \text{rarity}(g)$ is the exact cost incurred upon entering the goal state g .

2.

If $n = \text{goal}$:

Return 0

Different = number of positions where $n[i] \neq \text{goal}[i]$

Min_rarity = minimum rarity of any word in the dictionary

$c_{\min} = 1 + \text{min_rarity}$

Goal_cost = 1 + rarity(goal)

Return goal_cost + (different - 1) * c_min

3. Yes, this heuristic is admissible because it never guesses a cost that is higher than the real cheapest cost to reach the goal. Each move changes only one letter, so at least one move is needed for every different letter. The heuristic uses the exact cost of entering the goal and the cheapest possible cost for other required moves, so it can not overestimate and is admissible.

4. N/A

5.

Word Pair	Heuristic	Solution Cost	Nodes Expanded
WHY → ASK	h0	38.8554	755
WHY → ASK	Proposed	38.8554	634
BEAR → DUCK	h0	38.2402	720
BEAR → DUCK	Proposed	38.2402	301

The results support using the proposed heuristic because it returned the same solution cost while expanding fewer nodes on both problems.

AI Disclosure: I used ChatGPT GPT-5.6 Sol to help me better understand how heuristics work and admissibility. I used it to understand heuristic overestimation and come up with my conclusions for problems of that nature. My answers were written from my understanding. I also used it to understand the zip function in python and how we can use it to pair characters at the same positions.